RESEARCH PASS · BUNGIE, 1998 · ENGINE REFERENCE

Soulblighter Internals

How *Myth II* actually runs a battle behind the sprites: a **30 Hz integer simulation** driven entirely by player commands, where a soldier, an arrow and a severed head are all the same kind of object, and the random-number seed doubles as the multiplayer checksum.

---

00 — ORIENTATION

The shape of it

Five decisions do almost all the work in this engine. Everything else in this document is a consequence of one of them.

OBJE

One physical body type

Units, projectiles, scenery and severed limbs all reference the same 64-byte object tag: gravity, elasticity, terminal velocity, vitality, resistances. There is no separate "unit physics" and "debris physics".

1 / 30 S

Integer time, integer space

30 ticks per second, positions in 1/512 of a World Unit, fractions in 16.16 or 8.8 fixed point. No float touches gameplay. Floats appear only in fog density and reverb.

PRGR

Effects are projectiles

Explosions, gore, contrails and blood are not a particle system. They are recursive projectile groups spawning more projectiles, each with its own physics and optionally its own damage.

SEED

The RNG seed is the checksum

Out-of-sync detection compares the current random seed between players. Because nearly every decision draws from the LCG, the seed is a running hash of the entire divergence history.

RECO

Commands, never state

Only seven verbs command units, and only those cross the wire. A saved film is not a

recording — it is a recipe for re-simulating the match from a seed.

HOW TO READ THE CONFIDENCE MARKERS

`DOCUMENTED` — stated in Bungie's *Fear/Loathing* editor documentation, the shipped manual, a Bungie or Project Magma postmortem, or read directly off a reverse-engineered struct definition whose field names are Bungie's own.

`INFERRED` — community interpretation, or a reading assembled from patch notes and behaviour. Plausible, not attested.

`UNRESOLVED` — I could not establish this from any reachable source, and I am not guessing.

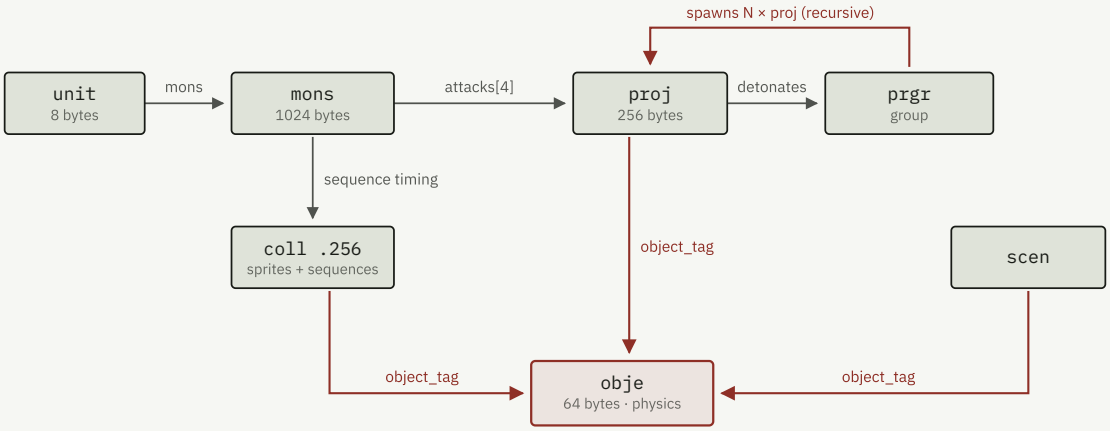
---

01 — DATA MODEL

The tag graph

Myth II's content is a flat pool of four-character-code tags in container files, resolved by ID at load time. Gameplay is almost entirely a question of which tag points at which.

The surprising part is the indirection depth. A `unit` tag — the thing a map marker places and a netgame roster counts — is **eight bytes**: a monster reference and a colour reference. Everything a player thinks of as "the unit" lives one level down in `mons`, and everything physical lives one level below that in `obje`.



A soldier, an arrow and a rock are the same kind of body. Gravity, elasticity, terminal velocity, vitality and per-damage-type resistance all live in `obje`, shared by monsters, projectiles and scenery — which is why an explosion can launch a corpse using the identical code path that launches debris. The accent path is the one that surprises people: a projectile's detonation names a *group*, which spawns further projectiles, each of which can detonate into another group. That recursion is Myth's entire explosion and gore system.

Two structural notes that catch out anyone reading the tags for the first time. DOCUMENTED

- **There is no `vitality` field on the monster tag.** Health is `obje.vitality_lower_bound` plus `vitality_delta`, rolled per instance — so no two soldiers have identical health. (In the original *Myth: The Fallen Lords*, `maximum_vitality` *was* on the monster; it moved down a level for Myth II.)
- **Monsters carry no damage numbers at all.** A unit's offensive power is entirely a property of the projectiles its attacks spawn. Rebalancing a warrior means editing an arrow.

---

02 — NUMERIC SUBSTRATE

Integer substrate

Every gameplay quantity in the engine is a scaled integer. This is the single decision that makes cross-platform lockstep possible, and it is enforced ruthlessly: the only IEEE floats anywhere in the gameplay tag set are `reverb_volume`, `reverb_decay_time`, `reverb_damping`, `fog_density` and `minimum_zoom_factor` — all presentation, none of it touching the simulation. DOCUMENTED

FIXED-POINT SCALE FACTORS

CODEC	DIVISOR	REPRESENTS	USED FOR
`World`	512	World Units (WU)	positions, velocities, radii, gravity, terminal velocity, knockback
`Fixed`	65536	16.16, 0.0–1.0	`absorbed_fraction`, `flinch_system_shock`, `berserk_vitality`, `miss_fraction`
`ShortFixed`	256	8.8	vitality, damage, scale, elasticity, mana, resistance multipliers
`Percent`	65535	probability	detonation frequency, contrail frequency, acceleration detonation
`ShortPercent`	255	probability, 1 byte	`healing_fraction`, terrain `movement_modifiers`
`Time`	30	seconds	every duration in the engine, stored as ticks
`Angle`	65536 / 360	degrees	facing; a full circle is 65536, so angle arithmetic wraps for free
`AngularVelocity`	65536 / 10800	degrees / second	`turning_speed`, `guided_turning_speed`

The 30 Hz tick is independently confirmed three ways: Project Magma's postmortem states it outright ("the base unit of time in Myth, equivalent to 1/30th of a second"); `ANGULAR_VELOCITY_SF` is literally defined as `65536 / (360 × 30)`; and a stock `turning_speed` of `0x071C` decodes to exactly 300.0 deg/sec under that scale, matching *Fear*'s stated units. `DOCUMENTED`

The lower-bound / delta idiom

Randomised ranges are everywhere — vitality, scale, initial velocity, ammunition, lifespan, animation rate, spawn counts, spawn positions, attack frequency, trigger delays. They are all encoded the same way, as a `X_lower_bound` plus an `X_delta`, and the delta has an off-by-one convention: the stored value is one greater than the true span when non-zero, so that "no randomness" and "a span of exactly zero" stay distinguishable.

This is how Myth gets its organic, never-quite-identical feel while staying bit-deterministic. Every roll comes from the seeded LCG in integer arithmetic, so the variation is identical on every machine.

The monster

The monster tag is 1024 bytes and reads like an inventory of every question the AI and combat systems ask about a unit. Grouped by what it actually controls:

MONSTER TAG — SELECTED FIELDS, WITH BUNGIE'S OWN DESCRIPTIONS WHERE THE EDITOR DOCUMENTS THEM

FIELD	GROUP	MEANING
`warning_distance`	Awareness	Distance at which the unit becomes aware of other units' *cries for help*. Stock ≈ 4 WU.
`critical_distance`	Awareness	Distance at which the unit is warned of another unit's presence; also the radius it searches when its target dies. Stock ≈ 8 WU.
`activation_distance`	Awareness	Distance at which the unit acts of its own volition on another unit.
`visibility_type`	Awareness	Field of vision; can be set to none, making a unit blind.
`pathfinding_radius`	Movement	The unit's "space bubble" — the minimum distance it keeps from an allied unit. ≈ 0.30 WU for infantry.
`base_movement_speed`	Movement	≈ 2.2 WU/sec for infantry.
`turning_speed`	Movement	Degrees per second.
`terrain_costs[16]`	Movement	Signed per-terrain route bias. Negative means impassable.
`movement_modifiers[16]`	Movement	Unsigned per-terrain speed scale. Zero means impassable.
`absorbed_fraction`	Defence	"The fraction of blows that a unit will absorb without damage."
`healing_fraction`	Defence	Fraction of health restorable by healing. 0.00 means healing kills — this is how undead are encoded.
`flinch_system_shock`	Reaction	How hard a hit must be to interrupt the unit. Trow are set very high.

FIELD	GROUP	MEANING
`hard_death_system_shock`	Reaction	Damage needed to trigger the violent death rather than the normal one.
`propelled_system_shock`	Reaction	Impact force needed to launch the unit. Requires the `CAN_BE_PROPELLED` flag.
`damage_to_propulsion`	Reaction	Propulsion speed and duration, 0–1.0.
`berserk_vitality`	Morale	Health fraction at which the unit goes berserk. Myrkridia sit at 0.25.
`berserk_system_shock`	Morale	Single-impact threshold for the same state.
`combined_power`	AI	The AI's own threat metric, used for target prioritisation.
`longest_range`	AI	Maximum attack distance. A bow is 24 WU.
`sequence_indexes[26]`	Animation	Indices into the sprite collection's sequence table. See below.

Animation gates gameplay, directly

This is the most consequential coupling in the whole engine and it is easy to miss. An attack's *duration is not stored on the attack*. It emerges from the sprite sequence: `frames_per_view × ticks_per_frame` sets the floor, and the attack's `recovery_time` adds on top. Lengthening a monster's swing animation directly nerfs its DPS. `DOCUMENTED`

More striking: **a unit can block if and only if it has a blocking sequence.** There is no "can block" flag — `sequence_indexes[5] > -1` is the capability check. Animation data *is* gameplay data here. `DOCUMENTED`

```
// the 26 sequence slots; 19–25 remain undocumented
 0 idle_1        1 moving       2 placeholder   3 pause        4 turning
 5 blocking      6 damage       7 pickup        8 head_shots   9 celebration
10 trans_1-2    11 idle_2      12 trans_2-1    13 running     14 ammo
15 holding      16 taunting    17 gliding      18 propelled
```

The extended-flags hack

Post-1.4 Myth ran out of room in the monster tag and did something remarkable: when the `USE_EXTENDED` flag is set, the **unused terrain-cost bytes are reinterpreted as extra fields**. The `sloped`, `grass` and `desert` slots become bitfields of behaviour flags; `steep` becomes the block-timer modifier; `rocky` becomes the run-speed percentage; `marsh`

becomes the veterancy cap; `snow` becomes charge range. `DOCUMENTED` (the decode is `INFERRED` from community reverse-engineering)

It is load-bearing rather than cosmetic — running speed (default +20%), block timing (default 30 ticks, tunable from 0.033 s to 5.2 s) and the veterancy cap (default 5 kills) are only tunable because of this overlay. The flags it unlocks are the interesting AI switches: `CHASES_ENEMIES_INTELLIGENTLY`, `MOVES_TO_POSITION_TO_SHOOT_UPHILL`, `WALKS_AROUND_LARGE_UNIT_BLOCKAGES`, `MIXES_MISSILE_MELEE_BEHAVIOUR`, `DOESNT_CLOSE_ON_TARGET`, `EXTRA_AGGRESSIVE_MISSILE_UNIT`, `WALKS_ON_THE_SURFACE_OF_WATER`.

---

04 — COMBAT

Damage

Damage is a 16-byte struct embedded in the projectile (and in the lightning tag). Every field of it is doing something:

```
ProjDmg — 16 bytes
type                short   DamageType
flags               ushort  16 behaviour bits
damage_lower_bound  short   ShortFixed  /256
damage_delta        short
radius_lower_bound  short   World       /512
radius_delta        short
rate_of_expansion   short   World  — blast grows over time
damage_to_velocity  short   World  — damage → knockback coefficient
```

Note what is *not* there: no minimum-damage radius, no falloff curve. Instead there is `rate_of_expansion`, which makes a blast a **growing sphere** rather than a static falloff. `INFERRED` — that is almost certainly why Myth explosions read as an expanding shockwave that catches units as it passes, and why a distant unit can be hit a tick or two after a near one.

Twelve damage types, nine resistance slots

The runtime damage types are `EXPLOSION`, `MAGICAL`, `HOLDING`, `HEALING`, `KINETIC`, `SLASHING_METAL`, `STONING`, `SLASHING_CLAWS`, `EXPLOSIVE_ELECTRICITY`, `FIRE`, `GAS`, `CHARM`. There is no separate holy/unholy type — that is `MAGICAL`. **Healing is a damage type**, not a separate system, which is exactly why Project Magma had to ship a fix for "too much negative (healing) damage could kill a unit". `DOCUMENTED`

Resistance lives on the object tag as nine `ShortFixed` multipliers — `slashing_damage`, `kinetic_damage`, `explosive_damage`, `electric_damage`, `fire_damage`, `paralysis_duration`, `stone`, `gas_damage`, `confusion` — where 1.000 is neutral and 0 is immunity. Note that `paralysis_duration` and `confusion` modulate *effect duration*, not damage. The 12→9 mapping is many-to-one and `UNRESOLVED`: both lists are known, the correspondence between them is not documented anywhere I could reach.

Absorption is probabilistic, not fractional

Despite the name, `absorbed_fraction` is read by the community as the *probability of shrugging off an attack entirely*, not a percentage of damage removed. `INFERRED` — but the supporting evidence is good: the monster tag carries separate `absorbed_impact_projectile_group_tag` and `blocked_impact_projectile_group_tag` effects, which only makes sense if absorption is a discrete per-hit outcome that can fire its own sound and spark.

Knockback is simulation, not decoration

The chain is fully explicit: `damage_to_velocity` converts damage into speed → `propelled_system_shock` and `damage_to_propulsion` gate and scale it per unit type → the `CAN_BE_PROPELLED` flag permits it → the body then flies under `obje.gravity`, bounces on `obje.elasticity`, and plays the dedicated `propelled` sequence.

The decisive proof is historical: when Project Magma added a propulsion feature in 1.5 using floating-point maths, it **desynced multiplayer**. A purely visual effect cannot desync a lockstep simulation. Flying bodies are in the deterministic sim. `DOCUMENTED`

"Ragdoll" is the wrong word, though. There is no articulated body — a propelled corpse is a single point mass with gravity and elasticity, playing one animation, skipping along the height field. That is precisely what it looks like.

Gore is a projectile group

Dismemberment is not a special system. `exploding_projectile_group_tag`, `dying_projectile_group_tag` and `burning_death_projectile_group_tag` each name a group that spawns a randomised count of debris projectiles at randomised offsets, each with its own `debris_sequence`, `bounce_sequence`, object physics, and flags like `IS_BLOODY`, `BLOODIES_LANDSCAPE` and `DETONATES_AT_REST`. Body parts are fully simulated projectiles — deterministic, counted against the projectile budget, and in principle capable of carrying damage. `DOCUMENTED`

Blood itself goes further. Bungie's TFL postmortem: "blood was made to destructively alter the terrain's color map." It is not a decal layer; it edits the landscape texture. Fire goes further still — `CELL_WAS_OR_IS_ON_FIRE_BIT` is a per-cell state bit *in the mesh*, so terrain fire spreads inside the simulation.

```
THE VISUAL / SIMULATION DIVIDING LINE
```

Two parallel effect systems exist. `prgr` — "projectile groups" — spawn real `proj` with damage and physics. `lpgr` — *local* projectile groups — carry no damage struct at all and reference their own separate `phys` ("Local Physics") tag, with their own budgets (raised in 1.7.2 from 512 to 768 local projectiles per map). Nearly every tag that can spawn a `prgr` also has an `lpgr` field beside it. `INFERRED`: "local" means client-local and non-networked — the sparks, splashes and dust that need not match across machines and can therefore run on cheaper, non-deterministic physics.

---

05 — ATTACKS

Ballistics

Every monster has up to four 64-byte attack definitions. The attack owns aiming, timing and veterancy; the projectile it names owns damage and flight.

```
AttackDef — 64 bytes, 4 per monster
flags                                 16 aiming/targeting bits
miss_fraction                         Fixed — flat chance to simply miss
projectile_tag
minimum_range / maximum_range         World
sequences[4]                          {flags, sequence_index, skip_fraction}
repetitions                           attacks carried out per command
initial_velocity_lower_bound / _delta World
initial_velocity_error                World — scatter; the de facto accuracy stat
recovery_time                         Time — seconds between attacks
recovery_time_experience_delta        Time — removed per kill
velocity_improvement_with_experience  World — removed per kill
mana_cost                             ShortFixed
```

The aiming behaviour is entirely flag-driven, and the flag names give away that the engine solves a real ballistic problem rather than hit-scanning:

- `LEADS_TARGET` — predictive aim at a moving target.
- `IS_INDIRECT` — arcing / mortar trajectory.
- `IS_FIXED_PITCH` — fixed launch angle; solve for speed instead.
- `DOES_NOT_REQUIRE_A_FIRING_SOLUTION` — the phrase "firing solution" is the engine's own, and its existence implies the projectile equation is actually being solved against `obje.gravity`.
- `AIMED_AT_TARGETS_FEET` — aim at the ground under the target, which for an area-of-effect projectile is strictly better.

- `DONT_SHOOT_OVER_NEARBY_UNITS`, `LOB_TO_HIT_LOWER_NEARBY_UNITS`, `AVOIDS_FRIENDLY_UNITS` — the engine tests the *arc* against intervening friendlies. Three separate flags exist just to govern shooting past allies, which tells you how much friendly fire mattered to the design.

Veterancy

Two experience terms sit on every attack: kills reduce `recovery_time` and reduce `initial_velocity_error`. Veterans therefore fire both faster and straighter. The cap is 5 kills by default, overridable through the extended-flags overlay. Kill credit also propagates — `experience_kill_bonus` is granted to allies within `experience_bonus_radius`, and `enemy_experience_kill_bonus` can grant it to nearby enemies too. DOCUMENTED

Wind

Projectiles carry an `AFFECTED_BY_WIND` flag and the mesh header names a `wind` tag, so wind is per-map, global, and opt-in per projectile. Whether it is a constant vector or time-varying is UNRESOLVED — no reachable source decodes that tag.

---

06 — MESH

World & collision

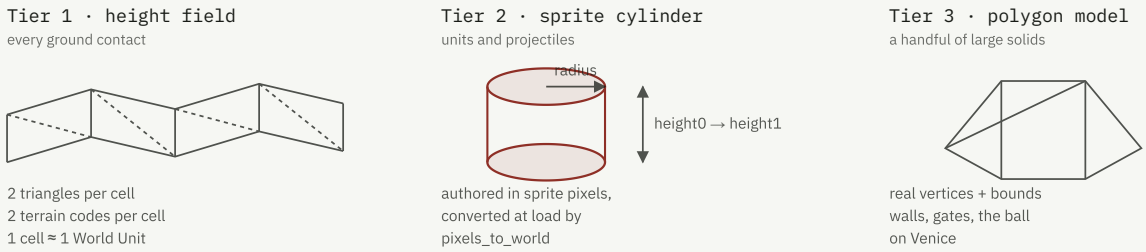
A Myth II map is a **regular grid height field**, not a polygon soup and not a navmesh. Bungie's own postmortem: "The game's terrain is a 3D polygonal mesh constructed from square cells, each of which is tessellated into two triangles." The tag confirms it exactly — each cell is 12 bytes carrying a height, two normals, flags, a packed terrain byte, a media height and a model index. DOCUMENTED

The terrain byte packs **two 4-bit terrain codes per cell**, one per triangle — so terrain type is stored at twice the resolution of the height grid. A submesh is 32×32 cells textured by a 256×256 colormap chunk, i.e. 8 colormap pixels per cell. Working from cell counts, stock maps land somewhere around 190–230 WU per side. INFERRED

THE 16 TERRAIN TYPES — A 4-BIT ENUM SHARED BY THE MESH AND BY EVERY MONSTER'S COST ARRAY

0–3 · MEDIA DEPTH	4–7	8–11	12–15
MEDIA_DWARF_DEPTH	SLOPED	ROCKY	LOATHING_SPECIAL
MEDIA_HUMAN_DEPTH	STEEP	MARSH	UNUSED
MEDIA_GIANT_DEPTH	GRASS	SNOW	WALKING_IMPASSABLE
MEDIA_DEEP	DESERT	FOREST	FLYING_IMPASSABLE

Water is a *terrain type*, not a volume — depth is quantised into four classes that gate which unit sizes may enter. Dwarf-depth passes most units; giant-depth admits Trow, large Myrkridia and dead bodies; deep media admits the dead and the amphibious. `FLYING_IMPASSABLE` is the map-edge fence. And because the `Mesh Cell Modifier` script action can rewrite terrain nibbles at runtime, **passability is mutable mid-level** — collapsing bridges and opening gates are terrain edits.



There is no general 3D collision. Ground contact is against the height field; unit-and-projectile intersection uses a vertical cylinder whose radius and height band are authored *in sprite pixels* and converted at load time via `pixels_to_world`; and a separate, harder polygon path handles a small number of large solid models. That third path is where Myth's famous collision bugs lived — Project Magma's changelog includes "improved precision in model collision detection code (fixes the Fetchball invisible wall bug)" and "fixed some cases where an object could pass through a model."

---

07 — MOVEMENT & AI

Movement & AI

Pathfinding is the one system Bungie filed under *What Went Wrong*, and they described the algorithm plainly:

> "First, we ignore all obstacles and calculate the A* path based solely on the terrain impassability... If the planned path would cause us to hit an obstacle, we need to deviate our path. We recursively consider both left and right deviations, and choose the direction that causes us to deviate least from our A* path."
>
> Bungie — Myth: The Fallen Lords postmortem

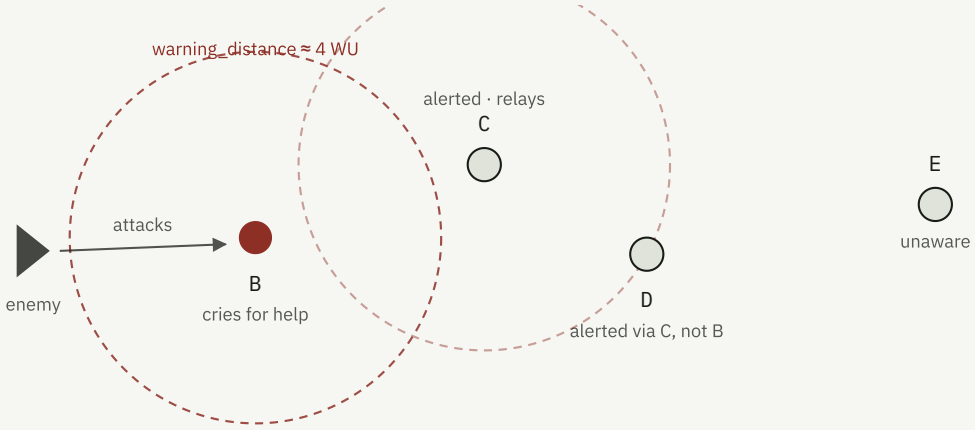
So: a coarse A* over *static* terrain cost only, then a periodic vector-based local refinement that branches recursively around dynamic obstacles and picks minimum deviation.

DOCUMENTED

That split is also a determinism win — A* over static terrain is trivially identical everywhere, and the local avoidance is a bounded recursive search with a deterministic tie-break. But it is exactly why units bunch and slide: `pathfinding_radius` is a purely *local* pairwise separation rule with no global crowd model, so dense groups compress rather than reflow. INFERRED, though the patch history supports it: 1.7.1 fixed the community-named "magic dirt" bug (a Trow half-surrounded by enemies, unable to escape an open route) and units becoming stuck together; 1.8.4/5 fixed "counter-intuitive source/destination pairings" in group moves, meaning formation slot assignment was originally not distance-optimal and generated crossing traffic. `USES_ANTICLUMP` remains an *opt-in* per-map and per-game flag, because turning it on changes combat outcomes.

Awareness is social, not just optical

The three awareness radii are not three sizes of the same idea. `critical_distance` is presence detection and doubles as the retarget radius when a unit's current target dies — which is why units re-engage locally instead of sprinting across the map after a kill. But `warning_distance` is defined by Bungie as the distance at which a unit hears *other units' cries for help*. That makes alarm a propagating chain through a formation rather than each unit independently spotting a threat.



Alarm spreads as a chain, not a radius. Each unit hears help-cries only within its own `warning_distance`, so a formation propagates alarm hop by hop — which is why a Myth II line wakes up in a wave rather than all at once, and why an isolated unit can be killed quietly a short distance from its allies.

Formations are a tag, not code

The `form` tag is a flat parameter block — row counts and separations for the three line types, a staggered offset, a box separation (with rows derived as `round(sqrt(n))`), a rabble separation plus x-bias, a shared encircle radius with two different arc angles, three vanguard separations, and a circle separation. Ten formations, bound to number keys 1–0:

Short Line, Long Line, Loose Line, Staggered Line, Box, Rabble, Shallow Encirclement, Deep Encirclement, Vanguard, Circle. DOCUMENTED

Crucially, a movement command carries the formation **by index**, not as resolved positions — the engine expands it deterministically on every machine — alongside an explicit `final_facing` angle. Shape and heading are decoupled, which is what makes drag-to-orient a single 16-bit number on the wire. Each formation also has a *reference point*, the anchor that lands under your cursor; 1.8.2 had to fix Vanguard so the tip rather than a point ahead of it sat at the click.

Ambient life is first-class

Not decoration: `AMBIENT_LIFE` is a monster class, markers carry `AMBIENT_LIFE` and `HUNTED_CREATURE` flags, and the script layer has dedicated `Chicken`, `Deer` and `Bear` action types (bears attack nearby non-bears). The full class list is `MELEE`, `MISSILE`, `SUICIDE`, `SPECIAL`, `HARMLESS`, `AMBIENT_LIFE`, `INVISIBLE_OBSERVER`.

---

08 — RECO

The command stream

Because films record commands rather than state, the film format *is* the order grammar. Every command is an 8-byte header — `size`, `verb`, `player_index`, `time` (an absolute tick) — plus a payload. Bungie's own grouping comments survive in the reverse-engineered enum, including the self-deprecating one on the community's later additions.

```
// used by the server when no one is giving orders (0-1)
NULL, SYNC
// monster commands (2-8)
GENERAL, MOVEMENT, TARGET, ATTACK_LOCATION, PICK_UP, RENAME, ROTATION
// player commands (9-14)
DROP_PLAYER, CHAT, DETACH, UNIT_ADJUSTMENT, SKETCH, ALLY
// commands intended only to encode information in saved games (15-16)
PRESET_SELECTION, SYNC_FORMATION
// extra commands hacked on by rank amateurs (17-20)
INVENTORY, INVENTORY_DROP, INVENTORY_PREV, INVENTORY_NEXT
FORCED_ENDGAME, ADD_PLAYER, PAUSE, GAME_SPEED, SAVE_LOADED,
REPLACE_PLAYER, SET_TEAM_CAPTAIN
```

Three things fall out of this that are worth sitting with.

Seven verbs command units. That is the entire tactical vocabulary. `GENERAL` covers the stateless one-shots — `STOP`, `SCATTER`, `RETREAT`, `SPECIAL_ABILITY`, `GUARD`, `TAUNT`,

`CELEBRATE`. The nuance lives in flags, not verbs: `TargetFlags` carries `CLOSE` (close distance with the target — the attack-in-place versus advance-to-contact distinction), `ATTACK_CLOSEST`, `ATTACK_INDIVIDUAL`, `DONT_OVERRIDE_LOCAL_AI`, `DONT_OVERRIDE_EXISTING_TARGET` and `PLAYER_INITIATED`.

There is no SELECT command. Selection never crosses the wire. Every unit command carries its own explicit list of 16-bit monster IDs. Band-select, double-click type-select (gated per unit type by `DOUBLE_CLICK_SELECTS_MONSTER_TYPE`), shift-add and preset groups are all pure client-side UI; only the resulting order is transmitted. `PRESET_SELECTION` exists solely to record group assignments into *saved games*. `DOCUMENTED`

Orders are not queued. A new command replaces the current one. The single exception is the waypoint array inside one `MOVEMENT` command: shift-click accumulates waypoints, and the `LOOP` / `LOOP_BACK_AND_FORTH` flags turn that array into a patrol. "Queuing" is a property of one order's payload, not of a scheduler.

And a design decision worth stealing: **map scripts inject the same verbs a human issues.** The `General Action` script type's command codes are byte-identical to the player `GeneralCommands` enum. Scripts have no private AI channel — which is exactly why `FROM_MAP_ACTION` and `PLAYER_INITIATED` flags exist to tell the two provenances apart, and why Myth II scripting composes so cleanly with player control.

---

09 — DETERMINISM

Determinism

The topology is client/server; the simulation is deterministic lockstep. The host is a command relay and arbiter, not a state authority.

> "All copies of Myth in a network game run deterministically and merely interpret the commands that they receive. This makes cheating difficult; if you hack the game to perform something illegal, such as making all your units invincible, you'll go out of sync with other players."
>
> Bungie — Myth: The Fallen Lords postmortem

The random seed is the checksum

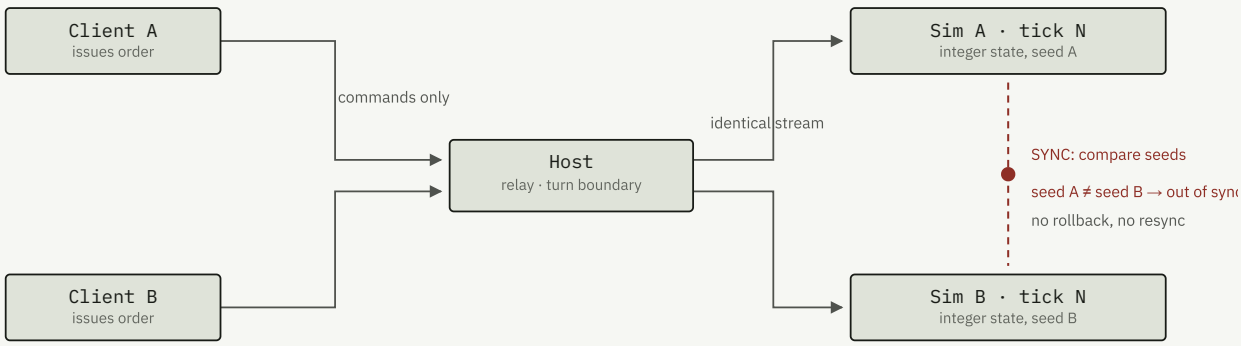
This is the cleverest thing in the engine. Project Magma: *"Myth detects 'Out of Sync' errors by periodically comparing the current random seeds of all players in the game to make sure they match."* `DOCUMENTED`

Myth does not hash the world. It compares a 32-bit RNG seed. Because virtually every non-deterministic decision draws from the generator, the seed is a running hash of the entire divergence history — if two machines have consumed a different *number* of draws, or fed the generator different values, their seeds separate immediately and never re-converge. Thirty-two bits per player per check, and it catches everything.

The generator itself is a textbook LCG, and it is worth writing out because the whole architecture rests on it:

```
seed' = (1664525 · seed + 1013904223) mod 2³²
result = (seed' >> 16) & 0xFFFF        // discard the weak low bits
```

Two seeds are stored, not one: a 32-bit `random_seed` for the world, and a separate 16-bit `initial_team_random_seed` used only for pre-game team composition and slot assignment. Splitting them means lobby shuffling never perturbs in-game RNG.
DOCUMENTED



Nothing but commands crosses the wire. Units are addressed by 16-bit index — "move units 12, 47, 93" is only meaningful if every machine agrees what unit 47 is — which is the signature of true lockstep. On mismatch there is no recovery path, and there cannot be: with no state on the wire, there is nothing to resynchronise from. Bungie's own phrasing was "a message will indicate that you're out of sync (and out of luck)."

The cross-platform bug, and how they found it

Mac and PC players desynced against each other while playing fine among themselves. Cause: "the floating point result of a division operation would contain a different number of significant digits after the decimal point on the x86 PC than on the PowerPC Mac."

The fix was to move all decimal logic to fixed integers. The *diagnosis* is the elegant part: Magma macro-replaced every `myth_random()` call with `debug_myth_random(__FILE__, __LINE__)`, logged every draw with its call site on both platforms, and diffed the logs. The

first divergent line named both the exact tick and the exact source line. That technique works precisely because the seed is the sync token. DOCUMENTED

Why films break between versions

A film is a recipe, not a recording — playback re-simulates the match from the stored seed. Any change to game logic makes the replay diverge, and it surfaces as the same out-of-sync error a live game would give. Hence a fixed 2,606-byte preamble carrying `version`, `buildnum`, and version-gate option flags (`USES_TFL`, `VERSION_151`, `PATCH_VERSION_0/1`, `USES_ANTICLUMP`) so the engine can re-enter the *old* rules on playback.

The breaks are syntactic as well as semantic. After 1.8.4 the `player_index` byte changed meaning from a roster index to a unique player identifier, and `USES_184_MATCHING` flags appear on movement and target commands to append a trailing centerpoint — old films must still replay under the old, worse formation-matching algorithm. Tag sets are checked too: `public_tags_checksum` and `private_tags_checksum` are compared at join, because a tag difference guarantees an immediate desync.

Films are correspondingly tiny — they scale with *orders issued*, not with match length or unit count, which is why Myth's film-sharing culture worked at all.

---

10 — LOATHING

Map scripting

Bungie's scripting story is a cautionary one — they filed *Scripting* under What Went Wrong, having abandoned a Java-based language mid-development when the responsible programmer left. What shipped instead is not a language at all. It is a **trigger graph**.

Actions live in the mesh tag as a fixed 64-byte header array plus a packed parameter blob. Each header carries an id, an expiration mode, a four-character type code, flags, a randomised trigger delay (`lower_bound` + `delta`, in ticks), a parameter count, and — charmingly — the editor's visual outline `indent`, persisted to disk. DOCUMENTED

Parameters are a self-describing TLV format with 21 types, including `WORLD_POINT_3D`, `MONSTER_IDENTIFIER`, `ACTION_IDENTIFIER` and — the escape hatch — `FIELD_NAME`, which lets one action name and write into *a field of another action*. That is how the `Mathematics` action returns results, and how the community's "param munging" works. Because parameters are self-describing, the editor can round-trip action types it does not understand.

Control flow is event-driven, not polled. Each action is a small state machine wired to others by edges: *activates on* success / failure / trigger / execution / activation /

deactivation / error / inhibition / missing prerequisites, plus *deactivates on* the same set, plus prerequisites, inhibitions and simultaneous activation. There is no loop construct, no variables beyond `FIELD_NAME` writes, and no expressions at all until `Mathematics` arrived in 1.7.0. Since `ACTION_IDENTIFIER` parameters can point anywhere, the script is a *graph*, not a tree.

Around 68 action types shipped, and the distribution tells you what Bungie actually spent scripting on: roughly half are unit-AI orchestration (`Melee`, `Harass`, `Surround`, `Rout`, `Squad`, `Platoon`, `Legion`, `Group Guard`, `Invisible Pursuit`, `Wandering Movement`), with a long tail of presentation, game-rule and archetype actions (`Assassin`, `Shaman`, `Peasant`, `Bear`). Two were documented but never implemented: `CD Audio` and `Connector`.

One more nice touch: the editor's entire action palette and help system is *generated from tags*. Each action type ships a `temp` string-list tag holding its display name, expiration mode and per-parameter documentation. Adding an action type to the engine means shipping a new tag, not rebuilding the editor.

---

11 — LIMITS OF THIS PASS

Open questions

Myth II's source has never been published — it went to MythDevelopers and Project Magma under licence. Everything above comes from Bungie's shipped editor documentation, two postmortems, Project Magma's changelogs, and a community reverse-engineering of the binary tag structures whose field names are recovered from the game itself. That leaves real holes, and it is worth naming them rather than papering over them.

- UNRESOLVED **Ticks per network turn, and input delay.** Commands are timestamped at tick granularity, so turn length is invisible in the file format, and nothing stores a negotiated latency value — suggesting a compile-time constant. Decoding `RecordingBlockHeader.flags` across real films, or measuring command-time gaps at block boundaries, would settle it.
- UNRESOLVED **Per-tick update ordering.** The order is certainly fixed and identical on every client — lockstep requires it — but no reachable source states it.
- UNRESOLVED **Render-side interpolation.** Neither postmortem discusses whether the renderer interpolates between the last two tick states or simply redraws the latest one.
- UNRESOLVED **The 12 damage types → 9 resistance slots mapping.** Both lists are known; the correspondence is not.
- UNRESOLVED **The `phys` ("Local Physics") and `wind` tag layouts**, and the meaning of `skip_fraction` on attack sequences, `hack_flags`, `visibility_type` values, and sequence indices 19–25.

Three leads would close most of this. Bungie released the **bungie.net metaserver source** (stripped but runnable) through The Tain — it will not contain the simulation, but it is the most likely public source for the join handshake and seed distribution. Road's **Map Action Script Guide (draft)** explicitly promises chapters on runtime architecture and AI orchestration. And the **MythingOak wiki** hosts tag documentation that hard-blocks automated fetching but opens fine in a browser.

---

12 — PROVENANCE

Sources

1. Postmortem: Bungie's *Myth: The Fallen Lords* — the terrain mesh, the two-tier pathfinding algorithm, the deterministic network model, destructive blood on the colour map, and the *Fear*/*Loathing* toolchain.
2. Postmortem: Project Magma's *Myth II* 1.5 and 1.5.1 — the 1/30 s tick, the fixed-integer conversion, seed comparison as OOS detection, and the `debug_myth_random` diagnosis.
3. jwheare/mythextract — reverse-engineered parsers for the tag, mesh, collection and recording formats. Every struct layout, flag enum, scale factor and command verb quoted above comes from here; the field names are Bungie's own, recovered from the binaries.
4. Fear documentation v1.8.5 and Loathing documentation v1.8.5 — Bungie's own editor manuals; the source for every quoted field description, the formation list and the map-action catalogue.
5. Project Magma release notes (1.7.1, 1.7.2, 1.8.0, 1.8.2–1.8.5) — engine behaviour changes, collision and pathfinding fixes, film compatibility work, and the projectile budget constants.
6. The Tain — source code archive, including Bungie's released bungie.net metaserver code, and The Tain's Myth II archive generally.
7. MythingOak — Category:Tags — confirmed to exist and to document these tags, but it rejects automated fetching; open it in a browser.